GSS religiosity and media pipeline: report

Yaqeen Institute data engineering assignment
2026-11-03

1. Extract and variable mapping

Cuts and years. Two GSS Data Explorer extracts: cut A = 2016 + 2018 (5,215 respondents), cut B = 2018 + 2021 (6,380). They share all of 2018 (2,348 respondents: 45% of A, 37% of B) and hold 9,247 distinct respondents. Three consecutive waves were chosen deliberately: 2016 and 2018 are the last in-person waves with every item of interest; 2021 is the first web-mode wave and lacks `postlife` and `reliten`. Including it exercises the "not asked that year" path for real, gives a three-wave series for question 6, and makes the mode change an explicit caveat. The alternative 2018 + 2022 (everything except `inttech`) would have hidden the problem the brief asks us to handle. The recipe is in the README; the files are committed byte-for-byte with a SHA-256 manifest.

Variable mapping.

question	variable(s)	years	universe / handling
Working status	`wrkstat`	all	full sample; grouped Working (1-3) / Unemployed (4) / Not in labour force (5-8)
Worked 12+ months	`evwork`	all	asked **only** of wrkstat 5-8; the brief's "if working" is inverted relative to the instrument. Unemployed shown as not asked, never pooled
Education	`degree`	all	full sample
Industry	`indus10`	all	271 Census codes; rolled up to NAICS sectors via a seed
Job satisfaction	`satjob`	all	working or ever worked; 2016 partly ballot-limited
Life after death	`postlife`	2016, 2018	**not fielded 2021**: shown as not asked, with n
Prayer frequency	`pray`	all	full sample
Religion raised in	`relig16`	all	13 categories grouped into 8 families via a seed
Newspaper; TV	`news`; `tvhours`	all	ballots a and b only
Interest in technology	`inttech`	all	ballot-limited; ballots a+b in 2016/18, b+c in 2021
required extras	`relig`, `attend`, `reliten` (2021: `relitenv`/`relitennv`), `wwwhr`, `age`, `sex`, `region`, `ballot`, `wtssps`, `wtssnrps`		`reliten` not fielded 2021; its two half-sample variants are modelled with validity flags

Cells where an item did not apply, was refused, unknown or not fielded carry a typed `value_status` (Section 4); they are never silent nulls and never counted as answers.

2. Data profile (written before any pipeline code; `docs/data_profile.md`)

Counts: 2016 = 2,867; 2018 = 2,348; 2021 = 4,032; no duplicate `(year, id_)` in either cut; the 2,348 overlapping rows are identical across cuts. Missing shares per variable and year are tabulated in the profile. Shapes: ballot-limited media items are about a third `inapplicable` every year; `evwork` is 62-65% `inapplicable` by design; `postlife` has 9-10% "don't know" in 2016/18 and 100% "not available" in 2021; 2021 shows web-mode effects (`.s Skipped on Web`, 2-8% more missing on `sex` and `age`).

Six findings about the files shaped the design: (1) the Explorer's `GSS.json` is a Python `repr`, not JSON; the valid file is `post_processing_output.json`. (2) Values are exported as **labels**; codes exist only in the embedded metadata. (3) Types are inconsistent (`wtssps` string, `wtssnrps` float; counts have labelled endpoints such as "89 or older"). (4) Cut B has an extra column, `prayer` (my mistake in the Explorer cart). (5) The DDI 3.1 codebook is broken (all variables share one id). (6) "Not fielded this year" is encoded two ways: `.y` for `postlife`/`reliten` in 2021 but `.i` for `relitenv`/`relitennv` in 2016/18, so the source code alone cannot be trusted for it.

3. Database and schema

DuckDB, embedded, via `dbt-duckdb`: runs on a clean laptop after one `uv sync`, needs no credentials, is columnar and fast at the shapes here (unpivot, group-by, pivot), and gives real `EXPLAIN ANALYZE` plans. Rejected: PostgreSQL in Docker (a server, volume and password for 9,247 rows; the design keeps the swap to a profile change), SQLite (no first-class dbt adapter, weak analytics), cloud warehouses (paid). Schemas mirror the layers: `raw` (loader-owned), `metadata` (seeds), `staging`, `intermediate`, `marts`, `dbt_test_failures`.

4. Model and structures

Grain and keys. Raw: one row per extract x respondent, key `(extract_name, year, id_)`, JSON row stored untouched. `id_` restarts at 1 each year, so the respondent key is `(year, id_)`, rendered `respondent_key = '<year>-<id_>'`. `fct_responses`: one row per respondent x variable (219,061). `fct_respondents`: one row per respondent (9,247), wide, generated by a macro from the variable seed. `dim_variables`: validity by year.

Layers. Staging unpivots the JSON generically (no variable is named in SQL). Intermediate applies the survey logic once, in long form: dedupe the overlap (asserting the copies are identical), map label to code through the seeds, classify, parse counts and weights, attach groupings. Marts expose the long fact, the wide table and the catalogue. Adding a variable means extracting it and adding seed rows; no model changes.

Encodings and missing values. Every cell has a `value_status` from a closed set: `answer`, `did_not_apply` (-100, skip pattern or other ballot), `dont_know` (-98), `no_answer` (-99, -97 web skip, -90), `not_asked_in_year`, `uncodable`, plus `missing_detail` for the finer source reason. `not_asked_in_year` is decided by the **variable-year validity seed**, with the source's `.y` code as a tested cross-check, because of finding (6). `value_code`, `value_label` and `value_numeric` are null unless `value_status = 'answer'`, enforced by a test. Metadata is six version-controlled seeds: variables; substantive code labels (375 rows, generated from the extracts' own metadata by a script); reserved codes and their semantics; variable-year validity; industry sector ranges; answer groupings. Models read the seeds; no code or label is repeated.

Naming (`docs/naming_conventions.md`): `stg_gss__*`, `int_gss__*`, `fct_*`/`dim_*`, `seed_gss__*`; GSS mnemonics verbatim with a closed suffix set (`_code`, `_label`, `_status`, `_group`); every `share_*` beside its `n_*`; singular tests `assert_<expectation>`.

Weights (bonus). `wtssps` (post-stratified, every year, mean 1 within year) is the analysis weight; `wtssnrps` is carried for comparison. Analyses report unweighted and weighted shares side by side; a test asserts `sum(wtssps) = n` per year.

5. Libraries and frameworks

Python 3.12; `duckdb 2.0.0`, `dbt-core 1.12.6`, `dbt-duckdb 1.11.1`, `dbt_utils 1.3.0`; `pytest`, `ruff`, `sqlfluff`; `uv` lockfile; Dockerfile over the same lockfile; GitHub Actions. Rejected: pandas (not needed), Airflow/Dagster/Prefect (five sequential steps on static data; a service would add operations without changing what runs), Great Expectations (dbt tests live next to the models), Terraform (nothing to provision locally; AWS design in Section 9).

6. Results and caveats (tables in `docs/analysis_results/`)

Overlap audit. A 5,215; B 6,380; both 2,348; distinct 9,247; `fct_respondents` has 9,247 rows and keys, so every result below counts each respondent once.

Q1 prayer by religion raised in (9,051 answered both; 196 excluded). Daily prayer: raised Muslim 76% (n = 38), Protestant 61% (4,589), Catholic 51% (2,928), Other Christian 46%, Eastern religions 41%, Other 41%,

None 32% (975), Jewish 21% (182); weighted within 3 points. Caveats: small families; pooled across three years including the web wave.

Q2 afterlife belief by year. 2016: 80.7% of 2,588 who answered yes/no (271 don't know, 8 no answer, 0 not asked); 2018: 81.0% of 2,123 (216 don't know); **2021: not asked, 4,032 respondents, share null.** Weighted 79.8% / 80.6%. Caveat: "don't know" (9%) is excluded from the denominator; counting it as non-belief gives 73%.

Q3 industry by education, working respondents (5,334). Less than high school: Construction (14%); High school: Health care and social assistance (14%; most common detailed industry Construction); Associate: Health care (23%); Bachelor's: Health care (15%); Graduate: Educational services (30%). Caveat: sector winners are 14-30% pluralities; the detailed winner is shown too.

Q4 TV hours vs job satisfaction (4,043; ballots a/b within the satjob universe). Mean daily hours: very satisfied 2.81 (1,839), moderately 2.76 (1,594), a little dissatisfied 2.86 (445), very dissatisfied 3.20 (165); medians 2 everywhere; weighted within 0.05. A weak gradient at the dissatisfied end, driven by 2021 (3.7 h, n = 78). Descriptive only.

Q5 ever worked 12+ months, not currently working. Not in labour force: 92.7% of 3,352 answered (2016 91.2%, 2018 90.6%, 2021 94.8%; weighted 88.4%). Unemployed: 467, **not asked**, shown separately. The question is answerable only for the universe the instrument defines.

Q6 media change by wave (ballots a/b). Daily newspaper reading 21.1% (2016, n = 1,885) -> 20.0% (2018, 1,559) -> 23.8% (2021, 2,696); never 36.5% -> 43.5% -> 37.1%. Mean TV hours 3.03 -> 2.94 -> 3.46 (weighted 2.78 -> 2.74 -> 3.38). Caveat: 2016->2018 moves are within ballot-subset noise; 2018->2021 coincides with the web-mode switch and the pandemic.

7. Limitations and mitigations

limitation	done	unsolved
`postlife`, `reliten` not fielded 2021	typed `not_asked_in_year` with counts; `reliten` 2021 via its half-sample variants	no 2021 afterlife figure; cross-mode affiliation strength is a different instrument
ballot-limited items (a third to a half missing)	`ballot` kept; validity seed names the ballots; a test proves no answers off-ballot; asked denominators	smaller n; `inttech` ballots differ 2018 vs 2021
`evwork` universe opposite to the brief's wording	documented; Q5 answered for the supported universe	"if working, ever worked 12 months" is unanswerable
271 industry codes	sector roll-up seed; both levels reported	pluralities of 14-30%
2021 web mode	web-skip kept distinct; per-year breakdowns; trend caveats	no mode adjustment possible
inconsistent "not fielded" encoding	metadata-driven validity with a cross-check test	validity rows are hand-curated
extra `prayer` column in cut B	loaded raw, logged, excluded via `is_modelled = false`	cuts are not column-identical

8. Efficiency

Materialization. Staging: views, except `stg_gss__responses` (table), because the JSON unpivot is the most expensive operation and has two consumers. Intermediate and marts: tables (seconds to rebuild; the pivot and five seed joins should not re-run per analyst query). At full scale (75,700 x 7,000 = 530 M cells) the long models become incremental by survey year, the wide table is built per year and unioned, and staging reads Parquet written by the loader.

Query plan (`docs/query_plans.md`). The most expensive query is the classification model (0.37 s; everything else under 30 ms). Two thirds of its plan is the unpivot (DuckDB de-correlates the lateral `unnest` into a hash join back onto the record plus a `json_extract_string` per cell). One change, materialising the staging unpivot, brings it to 0.32 s, replaces ten operators with one table scan, removes the duplicate work for the second consumer, and returns identical rows. Four other rewrites were measured and rejected,

including one 15% faster that silently drops cells of any column present in only one cut. The data is too small for dramatic numbers; the write-up explains how to show it at scale.

New extract. Add a raw folder, one `extracts.json` entry and validity rows for its years. The loader loads only that extract (atomic delete + insert keyed by extract name); others are untouched. dbt rebuilds downstream in seconds; with incremental models only the new year would be processed.

Deliberately not built. Incremental models, partitioning, an orchestration service, a Postgres container, Terraform, a Parquet path: right at 100x the data, wrong at 12k rows.

9. Orchestration, configuration, observability, production sketch

Orchestration. `source/pipeline/run.py` runs load -> seed (+ seed tests) -> build (models and tests in dependency order) -> analyses -> docs, stops at the first failure and writes a run manifest. Triggers: `make pipeline`, CI on every push, or adding an extract and running again. Failure is visible in the exit code, `logs/last_run.json`, `logs/pipeline.jsonl`, `raw.load_runs` and `dbt/target/run_results.json`. Re-running is always safe because every step is idempotent. A failing dbt test stops everything downstream of its model (`dbt build` semantics), so analyses never run on data that failed a test.

Configuration and secrets. `GSS_*`/`DBT_*` environment variables with working defaults (`.env.example`; `.env` git-ignored). `dbt/profiles.yml` is committed and contains only `env_var()` lookups; a server database would take its password the same way.

Observability. JSON-line logs (event, extract, run id, counts, file hash, validation findings); `raw.load_runs` as a queryable audit of every load; dbt run results and stored test failures; analysis outputs committed as Markdown so result changes show in diffs.

Production sketch and other sources (bonus, described). Extracts land immutable in S3 with the manifest; a Glue Python job validates and writes long-form Parquet partitioned by year; Athena is the dbt target with the same project; EventBridge triggers on new objects; IAM is read-only on raw and read/write on curated per role; cost is Athena scanned bytes, a few dollars a month at full GSS size. Other sources (Pew Religious Landscape Study, Pew or Nielsen media-use series, ARDA) would be ingested as separate raw folders with their own code-list and validity seeds and mapped to shared concepts in `dim_variables` with a `source_system` column, never merged at respondent level because the samples differ.

CI (bonus). GitHub Actions runs lint, loader tests, the manifest check and the full pipeline with all dbt tests on every push; a failure blocks the merge and uploads logs and run results.

10. Risk and complexity analysis

What breaks first: (1) the SQL unpivot, linear in cells and already two thirds of runtime at 273 k; at 530 M it is hours and must move into the loader (Parquet long form). (2) The hand-curated validity seed, which grows as variables x years (69 rows today; about 245 k for the cumulative file) and must be generated from NORC's codebook with hand rows only for exceptions. (3) Label-to-code lookup, which assumes stable labels; each new source needs its own code-label seed and a shared concept layer. What grows fastest: the long fact and the tests over it; mitigations are incremental models, partition pruning and running expensive singular tests on changed partitions only. I would write long-form Parquet in the loader, generate the validity metadata, and move to partitioned warehouse storage once any source exceeds a few million cells.

11. Assumptions and divergence

Assumptions (verified where possible): `(year, id_)` unique and stable across extracts (verified for 2018); exported labels equal the embedded code-list labels (0 unmapped); no sentinels in continuous variables (verified); ballot assignment is random; `wtssps` is right for all three years; the 2021 mode effect cannot be corrected here. Divergence from the design note (log in `docs/design_note.md`): staging unpivot became a table after measurement; groupings moved to their own seed so the generated seed can be regenerated; `_group` columns only where groups exist; the loader tolerates and logs unexpected columns rather than rejecting them; seeds load with `dbt build` so their tests run first.

12. AI use and human oversight

Tool. Claude Code (Anthropic), used as AI support.

What the AI did. Explained the brief; guided navigation of the Explorer; kept the variable-availability log from screenshots and pasted "summary by year" tables I supplied; proposed the year choice, variable mapping and mitigations for me to decide; wrote the profiling script, loader and tests, dbt project, analyses, orchestrator, Makefile, Dockerfile, CI, docs and the first draft of this report; ran and debugged everything locally.

Where a human corrected the AI. Its initial guesses that `news` ended in 2016 and `indus10` began in 2012 were wrong; the Explorer pages I captured corrected them. I added `prayer` instead of `pray`; the AI spotted the mismatch from the pasted code list, I removed it from cut A, but it stayed in the cart for cut B, which is why cut B has an extra column. I rejected the AI's 15% faster deduplication because its own check showed changed results.

Self-evaluation. Data research and extract construction about 60% me / 40% AI; design decisions 50/50 (AI proposed, I chose); code and documentation about 85% AI / 15% me (review, corrections, decisions). I can explain every file in the repository.

Time spent. About 8 hours (Explorer research and extracts 2 h; pipeline, dbt and SQL 4.5 h; report 1.5 h). With more time: derive validity metadata from NORC's codebook; quantify design effects for small groups; run the Docker path on a clean machine.

Appendix A. Reserved codes observed

code	label	value_status	occurrences
-100	`.i: Inapplicable`	did_not_apply	39,221
-70	`.y: Not available in this year`	not_asked_in_year	12,096
-99	`.n: No answer`	no_answer	814
-98	`.d: Do not Know/Cannot Choose`	dont_know	762
-97	`.s: Skipped on Web`	no_answer (detail skipped_on_web)	395

Plus 10,430 cells of `relitenv`/`relitennv` in 2016/2018 that the source codes as `.i` and the pipeline reclassifies as `not_asked_in_year` (detail `not_fielded_per_metadata`).

Appendix B. Test inventory

Loader (pytest, 12): load twice is identical; A-then-B equals B-then-A; overlap rows identical; every run recorded; Python-repr file rejected; ragged columns rejected; duplicate key rejected; undeclared year rejected; missing key column rejected; nested value rejected; unexpected column is a warning; a failed re-delivery leaves previous rows intact and records the failure.

dbt (101): seed integrity (31); generic model tests (62: keys, not-null, accepted values, ranges, “value columns null unless answer”, “answers have a value”); singular (8): overlap rows identical across cuts; no unmapped values; metadata and source agree on “not asked in year”; no answers off-ballot; `evwork` only for non-working; `wtssps` sums to n per year; respondent counts match the profile; `postlife` and `reliten` not-asked for every 2021 respondent.